score_forecasts

Li Shandross

3/25/2022

Load in truth data and forecasts

We load in the hospitalization truth data and our forecasts. (Note: The load forecast parentheses function wasn't working, so alternate code is written to achieve the same output.)

```
full_hosp_truth <- load_truth("HealthData",
                          "inc hosp",
                          temporal_resolution="daily",
                          data_location = "remote_hub_repo")

inc_hosp_targets <- paste(0:30, "day ahead inc hosp")
mon_fc_dates <- c(as.Date("2020-12-07") + weeks(0:29))
fips <- filter(hub_locations, geo_type == "state", population >= 500000) %>% pull(fips)

load("../data/loadedfc_scores_df.RData")
load("../data/baseline_fc_scores.RData")

# forecasts_baseline <- load_forecasts(models = "COVIDhub-baseline",
#                                          dates = mon_fc_dates,
#                                          date_window_size = 6,
#                                          locations = fips,
#                                          types = c("point","quantile"),
#                                          targets = inc_hosp_targets,
#                                          source = "zoltar",
#                                          verbose = FALSE,
#                                          as_of=NULL,
#                                          hub = c("US"))

# models <- c("Base_arima_4root", "Base_arima_noTrans",
#             "Multiple3_arima_4root", "Multiple3_arima_noTrans",
#             "Topmost8_arima_4root", "Topmost8_arima_noTrans")
#
# files <- rep("path", 30)
# for (i in 1:6) {
#   files <- list.files(path=paste("../data/", models[i], "/", sep=""), pattern=".csv", full.names=TRUE
#   for (j in 1:30) {
#     if(i + j != 2) {
#       df <- read_csv(files[j]) %>%
#         mutate(model = models[i])
#       forecasts <- rbind(forecasts, df)
#     } else {
```

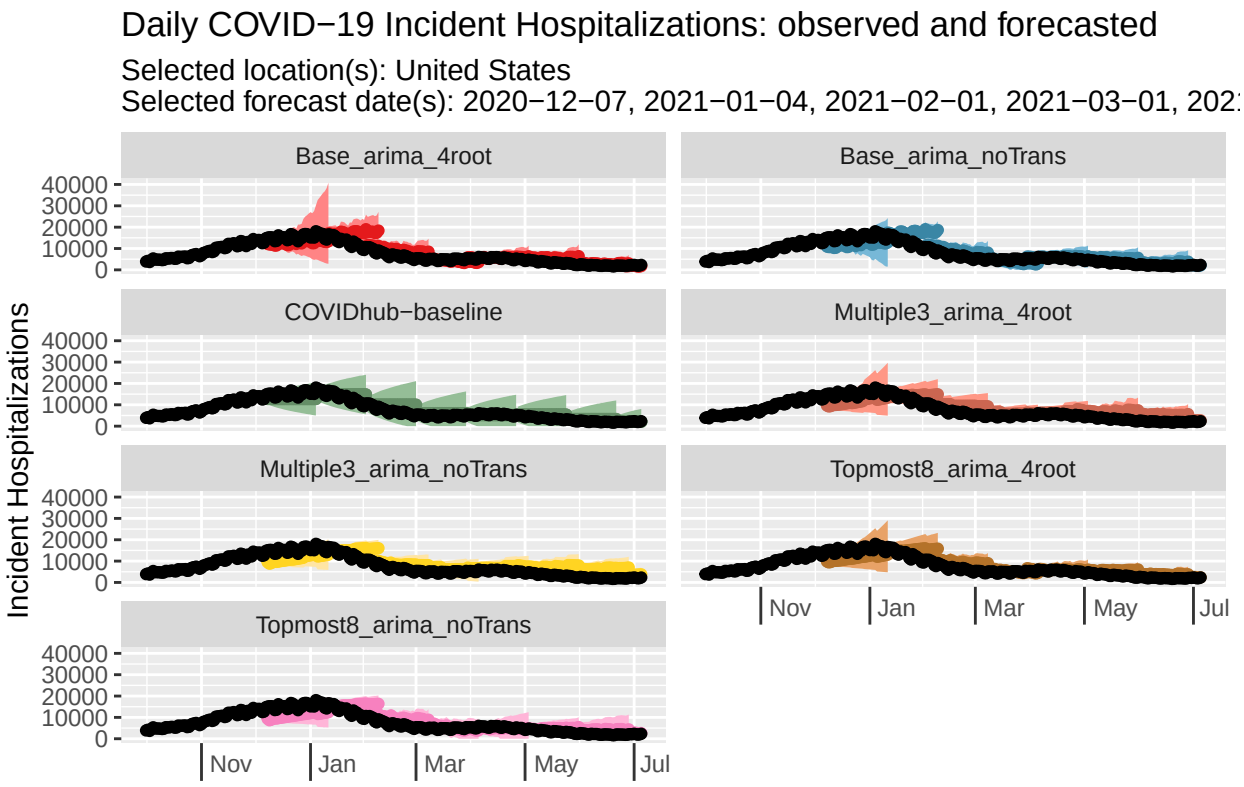
```
#         forecasts <- read_csv(files[j]) %>%
#         mutate(model = models[i])
#     }
#   }
# }
#
# forecasts_df <-
#   forecasts %>%
#     separate(target,
#       into=c("horizon", "temp"),
#       sep=" "
#     ) %>%
#     mutate(
#       horizon=as.numeric(horizon),
#       temporal_resolution="day",
#       target_variable="inc hosp"
#     ) %>%
#     select(model, forecast_date, location, horizon, temporal_resolution, target_variable, target_end_
#     left_join(hub_locations, by=c("location"="fips"))
#
# save(forecasts_df, file="../data/loadedfc_df.RData")
```

Plot Forecasts

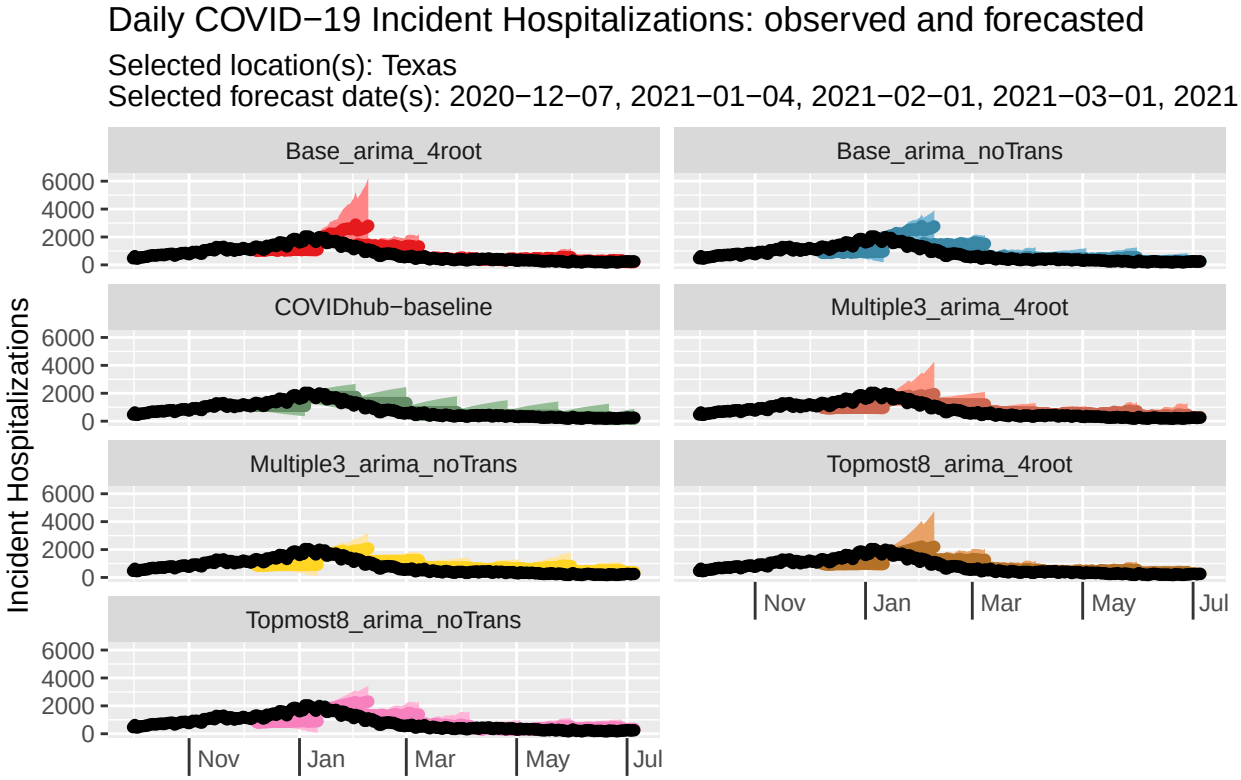
We plot forecasts for three locations (Us National, Texas, and Vermont) for each of the three models. The two state locations were chosen as the highest and lowest hospitalization count locations. The plotted forecasts are spaced 4 weeks apart with 1 - 28 day ahead horizons, starting with the first forecast date, 12/07/20, until the end of the training period, 07/05/21.

```
locs <- full_hosp_truth %>%
  filter(target_end_date <= as.Date("2021-07-05")) %>%
  group_by(location) %>%
  summarize(cum_value=sum(value)) %>%
  ungroup() %>%
  arrange(desc(cum_value)) %>%
  filter(row_number() %in% c(1, 2, 53)) %>%
  pull(location)

fc_dates <- c(as.Date("2020-12-07") + weeks(4*(0:7)))

fc_df <- forecasts_df %>% rbind(forecasts_baseline)
for (i in 1:3) {
  fdat <- fc_df %>%
    filter(location == locs[i], forecast_date %in% fc_dates)

  p <- plot_forecasts(fdat,
                      target_variable = "inc hosp",
                      intervals = c(.5, .95),
                      truth_source = "HealthData",
                      use_median_as_point = TRUE,
                      facet = model ~.,
                      facet_nrow = 4,
```

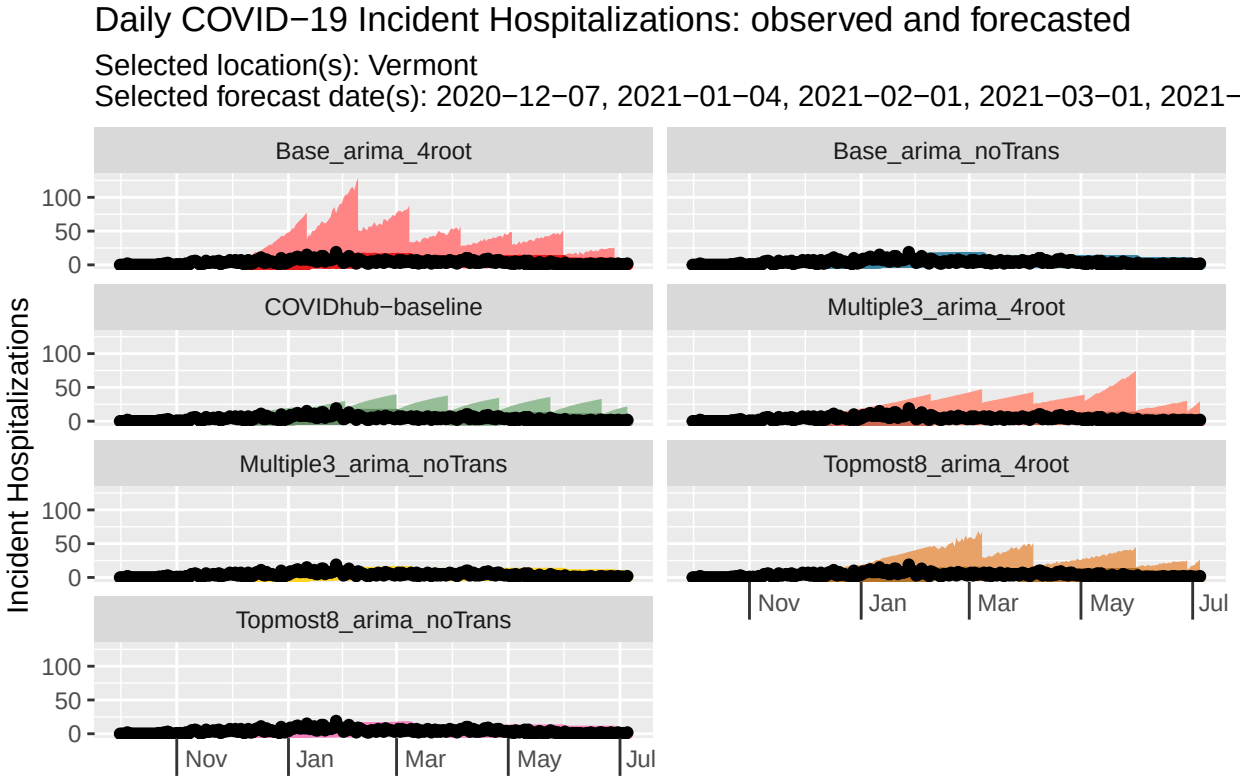
```
                        #facet_scales = "free_y",
                        fill_by_model = TRUE,
                        plot=FALSE)

  pt <- p +
    scale_x_date(name=NULL, limits=c(as.Date("2020-10-01"), as.Date("2021-07-05")),
                 date_breaks = "2 months", date_labels = "%b") +
    #scale_y_continuous(limits=c(0, max(filter(full_hosp_truth, location==locs[i])$value) * 1.25)) +
    theme(axis.ticks.length.x = unit(0.5, "cm"),
          axis.text.x = element_text(vjust = 7, hjust = -0.2),
          legend.position = "none")

  print(pt)
}
```



4root, Multiple3_arima_noTrans, Topmost8_arima_4root, Topmost8_arima_noTrans, COVIDhub−baseline (forecasts)

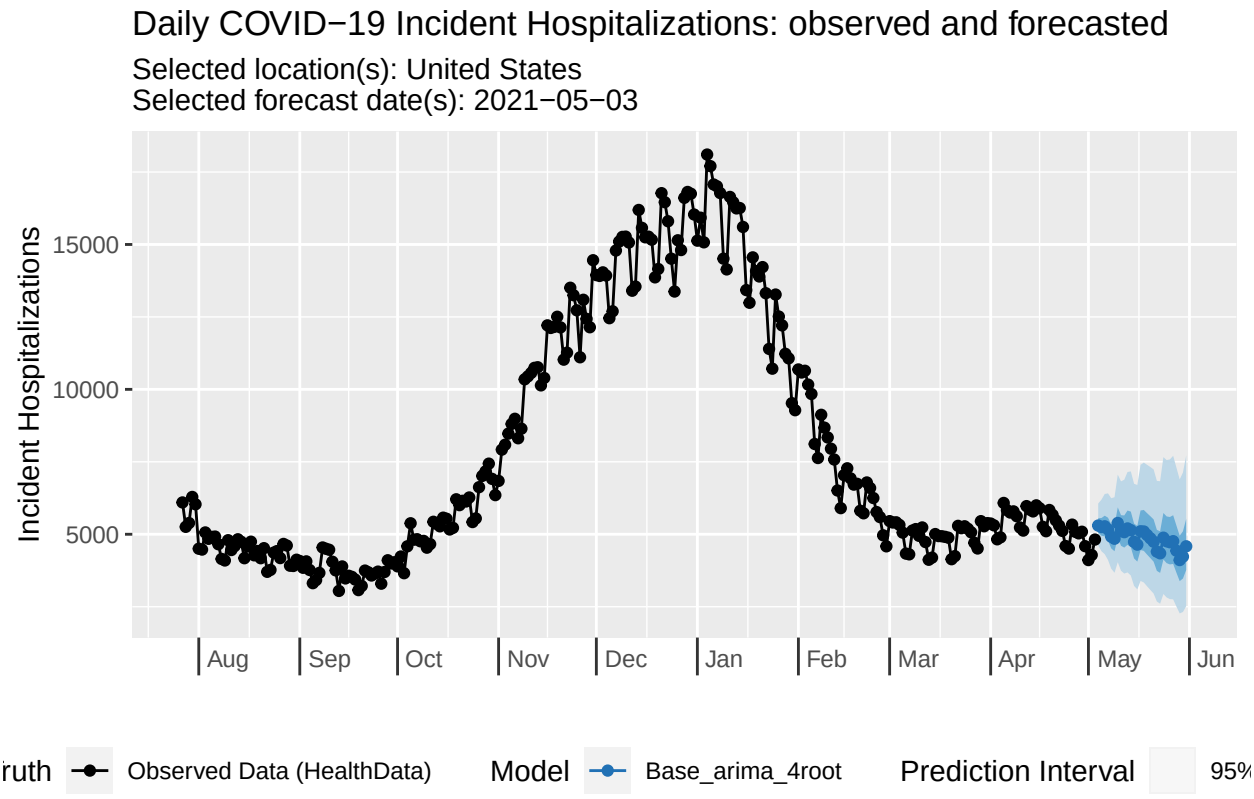


4root, Multiple3_arima_noTrans, Topmost8_arima_4root, Topmost8_arima_noTrans, COVIDhub−baseline (forecasts)



4root, Multiple3_arima_noTrans, Topmost8_arima_4root, Topmost8_arima_noTrans, COVIDhub−baseline (forecasts)

```
p <- plot_forecasts(filter(forecasts_df, model=="Base_arima_4root"),
                    truth_data = filter(full_hosp_truth, target_end_date <= as.Date("2021-05-03")),
                    locations="US",
                    forecast_dates=as.Date("2021-05-03"),
                    target_variable = "inc hosp",
                    intervals = c(.5, .95),
                    truth_source = "HealthData",
                    use_median_as_point = TRUE,
                    #facet = model ~.,
                    facet_nrow = 6,
                    #facet_scales = "free_y",
                    fill_by_model = TRUE,
                    plot=FALSE)

  pt <- p +
    scale_x_date(name=NULL, limits=c(as.Date("2020-07-27"), as.Date("2021-05-31")),
              date_breaks = "1 months", date_labels = "%b") +
    #scale_y_continuous(limits=c(0, max(filter(full_hosp_truth, location==locs[i])$value) * 1.25)) +
    theme(axis.ticks.length.x = unit(0.5, "cm"),
          axis.text.x = element_text(vjust = 7, hjust = -0.2),
          legend.position = "bottom")

  print(pt)
```



The plots don't show much difference between the 6 models (although this is partially due to the fact that the yscales are free and are not consistent even for the same location). However, of note is the fact that the models that use a forth root transformation tend to have wider prediction intervals than the models without transformations. All of the models reasonably follow the trend of the truth lines for every location shown, although it seems like lower count locations may have slightly greater accuracy. We investigate model accuracy more deeply by scoring the forecasts for each model and calculating appropriate metrics.

Score forecasts

We calculate metrics for all three models, aggregating by horizon. Since hospitalizations are forecasted at a daily time scale, a new horizon week variable is created in which scoring metrics are averaged over from a single week Tuesday to the following Monday. We perform these calculations for the entire training phase as well as for each of the two waves that occurred during the training phase.

```
scores_df <- score_forecasts(forecasts=forecasts_df, return_format="wide", truth=full_hosp_truth, use_m
#getwd()
#save(scores_df, file="../data/loadedfc_df.RData")
# score_baseline <- score_forecasts(forecasts=forecasts_baseline, return_format="wide", truth=full_hosp
#save(forecasts_baseline, score_baseline, file="../data/baseline_fc_scores.RData")
```

Overall metrics

```
# US National
overall_metrics_US <- scores_df %>%
  rbind(score_baseline) %>%
  filter(horizon <= 28,
         target_end_date <= as.Date("2021-07-05"),
         location == "US") %>%
  mutate(
    horizon_wk = case_when(
      horizon %in% c(1:7) ~ 1,
      horizon %in% c(8:14) ~ 2,
      horizon %in% c(15:21) ~ 3,
      horizon %in% c(22:28) ~ 4
    )
  ) %>%
  group_by(model, horizon_wk) %>%
  summarize(
    wis = mean(wis), mae=mean(abs_error),
    coverage_50 = mean(coverage_50),
    coverage_95 = mean(coverage_95)
  )
```

```
## `summarise()` has grouped output by 'model'. You can override using the `.groups` argument.
```

```
knitr::kable(overall_metrics_US)
```

model	horizon_wk	wis	mae	coverage_50	coverage_95
Base_arima_4root	1	542.0758	836.7515	0.1166667	0.8000000
Base_arima_4root	2	860.1603	1323.9434	0.3931034	0.8413793
Base_arima_4root	3	1156.8787	1765.0926	0.3367347	0.8265306
Base_arima_4root	4	1567.6260	2346.3728	0.2592593	0.7724868
Base_arima_noTrans	1	716.2359	1063.4244	0.0833333	0.7833333
Base_arima_noTrans	2	958.0868	1359.2817	0.4758621	0.8275862
Base_arima_noTrans	3	1200.9821	1718.6466	0.4897959	0.8469388
Base_arima_noTrans	4	1551.4425	2292.9678	0.3703704	0.8571429
COVIDhub-baseline	1	604.1557	773.9000	0.8000000	0.9666667
COVIDhub-baseline	2	1046.6620	1546.7172	0.7862069	1.0000000
COVIDhub-baseline	3	1287.1225	1930.4337	0.7551020	1.0000000
COVIDhub-baseline	4	1566.5810	2385.8042	0.7142857	1.0000000
Multiple3_arima_4root	1	1647.7722	2105.8313	0.0500000	0.2500000
Multiple3_arima_4root	2	1396.4283	2135.8242	0.1448276	0.6275862
Multiple3_arima_4root	3	1416.4636	2197.3083	0.2244898	0.7091837
Multiple3_arima_4root	4	1763.8124	2694.8007	0.2328042	0.5820106
Multiple3_arima_noTrans	1	1722.9155	2232.1456	0.0833333	0.3166667
Multiple3_arima_noTrans	2	1430.4456	2300.8614	0.0965517	0.7655172
Multiple3_arima_noTrans	3	1482.0334	2403.1234	0.2193878	0.8520408
Multiple3_arima_noTrans	4	1906.5482	3012.1596	0.2010582	0.7619048
Topmost8_arima_4root	1	1116.0340	1512.2554	0.0666667	0.3333333
Topmost8_arima_4root	2	1044.1200	1653.1901	0.2344828	0.7586207
Topmost8_arima_4root	3	1212.1617	1919.1659	0.2500000	0.7551020
Topmost8_arima_4root	4	1470.9169	2299.8640	0.2539683	0.6825397
Topmost8_arima_noTrans	1	1056.7760	1442.9441	0.2000000	0.7000000

model	horizon_wk	wis	mae	coverage_50	coverage_95
Topmost8_arima_noTrans	2	1129.0804	1778.4993	0.3241379	0.8137931
Topmost8_arima_noTrans	3	1298.1209	2080.3621	0.3214286	0.8469388
Topmost8_arima_noTrans	4	1588.5418	2432.3511	0.3809524	0.8306878

```
overall_metrics_US <- overall_metrics_US %>%
  mutate(
    rwis = case_when(
      horizon_wk == 1 ~ wis / pull(overall_metrics_US[9,3]),
      horizon_wk == 2 ~ wis / pull(overall_metrics_US[10,3]),
      horizon_wk == 3 ~ wis / pull(overall_metrics_US[11,3]),
      horizon_wk == 4 ~ wis / pull(overall_metrics_US[12,3])
    ),
    rmae = case_when(
      horizon_wk == 1 ~ mae / pull(overall_metrics_US[9,4]),
      horizon_wk == 2 ~ mae / pull(overall_metrics_US[10,4]),
      horizon_wk == 3 ~ mae / pull(overall_metrics_US[11,4]),
      horizon_wk == 4 ~ mae / pull(overall_metrics_US[12,4])
    )
  )

# Aggregated for all states and territories
overall_metrics_states <- scores_df %>%
  rbind(score_baseline) %>%
  filter(horizon <= 28,
         target_end_date <= as.Date("2021-07-05"),
         location != "US") %>%
  mutate(
    horizon_wk = case_when(
      horizon %in% c(1:7) ~ 1,
      horizon %in% c(8:14) ~ 2,
      horizon %in% c(15:21) ~ 3,
      horizon %in% c(22:28) ~ 4
    )
  ) %>%
  group_by(model, horizon_wk) %>%
  summarize(
    wis = mean(wis), mae=mean(abs_error),
    coverage_50 = mean(coverage_50),
    coverage_95 = mean(coverage_95)
  )
```

```
## `summarise()` has grouped output by 'model'. You can override using the `.groups` argument.
```

```
knitr::kable(overall_metrics_states)
```

model	horizon_wk	wis	mae	coverage_50	coverage_95
Base_arima_4root	1	17.82155	25.56775	0.5035256	0.8916667
Base_arima_4root	2	24.22446	37.39429	0.4862069	0.9278515
Base_arima_4root	3	31.05689	47.77079	0.4797881	0.9233713

model	horizon_wk	wis	mae	coverage_50	coverage_95
Base_arima_4root	4	39.58287	60.27310	0.4624542	0.9059829
Base_arima_noTrans	1	20.33340	28.78318	0.5227564	0.9025641
Base_arima_noTrans	2	26.07601	39.36830	0.5771883	0.9275862
Base_arima_noTrans	3	32.44306	48.42299	0.5897763	0.9182692
Base_arima_noTrans	4	40.14934	59.56371	0.5810948	0.9055759
COVIDhub-baseline	1	18.60346	23.20929	0.8371795	0.9756410
COVIDhub-baseline	2	30.32758	39.05756	0.8957560	0.9937666
COVIDhub-baseline	3	36.41161	47.92347	0.8958006	0.9928375
COVIDhub-baseline	4	43.01834	58.50152	0.8908221	0.9919617
Multiple3_arima_4root	1	35.49139	47.14203	0.3173077	0.7198718
Multiple3_arima_4root	2	34.46300	49.26281	0.3602122	0.8525199
Multiple3_arima_4root	3	45.00624	52.95093	0.3924647	0.8843210
Multiple3_arima_4root	4	90.81645	63.98867	0.3803419	0.8734229
Multiple3_arima_noTrans	1	35.15434	47.84168	0.3153846	0.7615385
Multiple3_arima_noTrans	2	33.42352	52.82432	0.3720159	0.9287798
Multiple3_arima_noTrans	3	36.46621	58.69364	0.4084576	0.9398548
Multiple3_arima_noTrans	4	44.74083	72.38728	0.3937729	0.9313187
Topmost8_arima_4root	1	26.96335	37.19601	0.3823718	0.7884615
Topmost8_arima_4root	2	26.54044	41.03636	0.4144562	0.9018568
Topmost8_arima_4root	3	30.69035	47.89070	0.4245487	0.9144427
Topmost8_arima_4root	4	36.82161	57.17866	0.4252137	0.9078144
Topmost8_arima_noTrans	1	25.96275	35.41195	0.4637821	0.8618590
Topmost8_arima_noTrans	2	27.77819	41.74064	0.5312997	0.9269231
Topmost8_arima_noTrans	3	32.35671	49.56238	0.5323783	0.9276884
Topmost8_arima_noTrans	4	37.99779	58.30459	0.5484330	0.9189052

```
overall_metrics_states <- overall_metrics_states %>%
  mutate(
    rwis = case_when(
      horizon_wk == 1 ~ wis / pull(overall_metrics_states[9,3]),
      horizon_wk == 2 ~ wis / pull(overall_metrics_states[10,3]),
      horizon_wk == 3 ~ wis / pull(overall_metrics_states[11,3]),
      horizon_wk == 4 ~ wis / pull(overall_metrics_states[12,3])
    ),
    rmae = case_when(
      horizon_wk == 1 ~ mae / pull(overall_metrics_states[9,4]),
      horizon_wk == 2 ~ mae / pull(overall_metrics_states[10,4]),
      horizon_wk == 3 ~ mae / pull(overall_metrics_states[11,4]),
      horizon_wk == 4 ~ mae / pull(overall_metrics_states[12,4])
    )
  )
```

When splitting the results so that US National is separate, we find that for US National the two models that use the base set hierarchy tended to perform the best, followed by the Topmost8_arima_noTrans model. The Base_arima_noTrans model performed the best in terms of PI Coverage, followed by the Base_arima_4root and Topmost8_arima_noTrans models (with the latter being more consistent with coverage rates for all horizons). In terms of WIS and MAE, the Base_arima_4root model performed the best, followed by the Base_arima_noTrans model. Overall, for the US National level, it seems that there is a pattern that for coverage rates, the models without transformations perform better (and these tend to have smaller prediction intervals) while for WIS and MAE the fourth root transform models perform better; these patterns hold at all horizons. Further, there is a general order to model accuracy in terms of hierarchical structure used: the Base set performs the best, followed by the Topmost8 set, and finally the Multiple3 set.

For the state level, we see noticeably better coverage rates that approach the nominal level (or even exceed it) for all of the models. (We also see shrunken WIS and MAE values, although this is likely partially due to the fact that the states have smaller scale values than the National level.) It's a little harder to definitively say which models have the best coverage rates, as the results differ between the 50% and 95% intervals and the models are quite similar in coverage rates, especially for the 95% intervals. However, it seems that the Base_arima_4root model has the best performance for all metrics across all horizons. It is followed closely by the Topmost8_arima_noTrans and Base_arima_noTrans models, then the Topmost8_arima_4root and Multiple3_arima_noTrans models, all of which are much better than the Multiple3_arima_4root model.

Metrics by wave

```
# US National
wave_metrics_US <- scores_df %>%
  rbind(score_baseline) %>%
  filter(horizon <= 28,
         target_end_date <= as.Date("2021-07-05"),
         location == "US") %>%
  mutate(
    horizon_wk = case_when(
      horizon %in% c(1:7) ~ 1,
      horizon %in% c(8:14) ~ 2,
      horizon %in% c(15:21) ~ 3,
      horizon %in% c(22:28) ~ 4
    ),
    wave = case_when(
      forecast_date <= as.Date("2021-03-17") ~"winter21",
      forecast_date >= as.Date("2021-03-17") ~"alpha"
    )
  ) %>%
  group_by(wave, model, horizon_wk) %>%
  summarize(
    wis = mean(wis), mae=mean(abs_error),
    coverage_50 = mean(coverage_50),
    coverage_95 = mean(coverage_95)
  )
```

```
## `summarise()` has grouped output by 'wave', 'model'. You can override using the `.groups` argument.
```

```
knitr::kable(wave_metrics_US)
```

wave	model	horizon_wk	wis	mae	coverage_50	coverage_95
alpha	Base_arima_4root	1	298.2706	478.3680	0.1000000	0.8333333
alpha	Base_arima_4root	2	483.6113	757.3713	0.4000000	0.9142857
alpha	Base_arima_4root	3	647.7496	1044.3615	0.3186813	0.9010989
alpha	Base_arima_4root	4	827.5967	1349.3859	0.2261905	0.8571429
alpha	Base_arima_noTrans	1	364.6295	648.5070	0.1000000	1.0000000
alpha	Base_arima_noTrans	2	563.0216	863.5479	0.5142857	1.0000000
alpha	Base_arima_noTrans	3	689.8029	1088.3635	0.5494505	1.0000000
alpha	Base_arima_noTrans	4	897.8446	1508.1117	0.4404762	1.0000000
alpha	COVIDhub-baseline	1	371.5489	323.9333	1.0000000	1.0000000

wave	model	horizon_wk	wis	mae	coverage_50	coverage_95
alpha	COVIDhub-baseline	2	774.3104	923.1429	0.9857143	1.0000000
alpha	COVIDhub-baseline	3	932.4429	1169.4286	0.9890110	1.0000000
alpha	COVIDhub-baseline	4	1113.1295	1492.3571	0.9761905	1.0000000
alpha	Multiple3_arima_4root	1	1400.6730	1766.6022	0.0000000	0.2000000
alpha	Multiple3_arima_4root	2	1224.0003	1771.3322	0.1714286	0.5714286
alpha	Multiple3_arima_4root	3	1391.1703	2040.6824	0.1538462	0.4945055
alpha	Multiple3_arima_4root	4	1782.3330	2639.0030	0.1190476	0.3571429
alpha	Multiple3_arima_noTrans	1	1620.3403	2151.7135	0.0666667	0.2000000
alpha	Multiple3_arima_noTrans	2	1472.2298	2394.2733	0.1000000	0.7428571
alpha	Multiple3_arima_noTrans	3	1692.6105	2754.0069	0.1538462	0.8901099
alpha	Multiple3_arima_noTrans	4	2187.8663	3535.4044	0.0595238	0.7380952
alpha	Topmost8_arima_4root	1	608.2186	882.1857	0.0000000	0.3333333
alpha	Topmost8_arima_4root	2	699.1666	1062.5035	0.2857143	0.6857143
alpha	Topmost8_arima_4root	3	887.1828	1363.9112	0.2197802	0.6703297
alpha	Topmost8_arima_4root	4	1072.7247	1672.5638	0.1904762	0.6190476
alpha	Topmost8_arima_noTrans	1	503.7217	841.2955	0.2333333	0.9333333
alpha	Topmost8_arima_noTrans	2	672.8638	1170.0135	0.4571429	1.0000000
alpha	Topmost8_arima_noTrans	3	848.2689	1468.6578	0.4835165	1.0000000
alpha	Topmost8_arima_noTrans	4	1011.0640	1649.8331	0.4761905	1.0000000
winter21	Base_arima_4root	1	785.8809	1195.1350	0.1333333	0.7666667
winter21	Base_arima_4root	2	1211.6061	1852.7441	0.3866667	0.7733333
winter21	Base_arima_4root	3	1598.1239	2389.7263	0.3523810	0.7619048
winter21	Base_arima_4root	4	2159.6493	3143.9623	0.2857143	0.7047619
winter21	Base_arima_noTrans	1	1067.8424	1478.3418	0.0666667	0.5666667
winter21	Base_arima_noTrans	2	1326.8143	1821.9665	0.4400000	0.6666667
winter21	Base_arima_noTrans	3	1644.0041	2264.8919	0.4380952	0.7142857
winter21	Base_arima_noTrans	4	2074.3209	2920.8526	0.3142857	0.7428571
winter21	COVIDhub-baseline	1	836.7624	1223.8667	0.6000000	0.9333333
winter21	COVIDhub-baseline	2	1300.8569	2128.7200	0.6000000	1.0000000
winter21	COVIDhub-baseline	3	1594.5116	2589.9714	0.5523810	1.0000000
winter21	COVIDhub-baseline	4	1929.3423	3100.5619	0.5047619	1.0000000
winter21	Multiple3_arima_4root	1	1894.8715	2445.0603	0.1000000	0.3000000
winter21	Multiple3_arima_4root	2	1557.3611	2476.0167	0.1200000	0.6800000
winter21	Multiple3_arima_4root	3	1438.3845	2333.0508	0.2857143	0.8952381
winter21	Multiple3_arima_4root	4	1748.9959	2739.4389	0.3238095	0.7619048
winter21	Multiple3_arima_noTrans	1	1825.4906	2312.5778	0.1000000	0.4333333
winter21	Multiple3_arima_noTrans	2	1391.4470	2213.6769	0.0933333	0.7866667
winter21	Multiple3_arima_noTrans	3	1299.5333	2099.0243	0.2761905	0.8190476
winter21	Multiple3_arima_noTrans	4	1681.4937	2593.5638	0.3142857	0.7809524
winter21	Topmost8_arima_4root	1	1623.8494	2142.3250	0.1333333	0.3333333
winter21	Topmost8_arima_4root	2	1366.0764	2204.4976	0.1866667	0.8266667
winter21	Topmost8_arima_4root	3	1493.8100	2400.3867	0.2761905	0.8285714
winter21	Topmost8_arima_4root	4	1789.4707	2801.7041	0.3047619	0.7333333
winter21	Topmost8_arima_noTrans	1	1609.8303	2044.5927	0.1666667	0.4666667
winter21	Topmost8_arima_noTrans	2	1554.8825	2346.4193	0.2000000	0.6400000
winter21	Topmost8_arima_noTrans	3	1687.9927	2610.5057	0.1809524	0.7142857
winter21	Topmost8_arima_noTrans	4	2050.5241	3058.3656	0.3047619	0.6952381

```
# Aggregated for all states and territories
wave_metrics_states <- scores_df %>%
  rbind(score_baseline) %>%
  filter(horizon <= 28,
```

```
        target_end_date <= as.Date("2021-07-05"),
        location != "US") %>%
  mutate(
    horizon_wk = case_when(
      horizon %in% c(1:7) ~ 1,
      horizon %in% c(8:14) ~ 2,
      horizon %in% c(15:21) ~ 3,
      horizon %in% c(22:28) ~ 4
    ),
    wave = case_when(
      forecast_date <= as.Date("2021-03-17") ~"winter21",
      forecast_date >= as.Date("2021-03-17") ~"alpha"
    )
  ) %>%
  group_by(wave, model, horizon_wk) %>%
  summarize(
    wis = mean(wis), mae=mean(abs_error),
    coverage_50 = mean(coverage_50),
    coverage_95 = mean(coverage_95)
  )
```

```
## `summarise()` has grouped output by 'wave', 'model'. You can override using the `.groups` argument.
```

```
knitr::kable(wave_metrics_states)
```

wave	model	horizon_wk	wis	mae	coverage_50	coverage_95
alpha	Base_arima_4root	1	8.305589	12.73108	0.5115385	0.9064103
alpha	Base_arima_4root	2	12.789024	19.85982	0.5043956	0.9464286
alpha	Base_arima_4root	3	16.393942	25.75905	0.5021133	0.9547760
alpha	Base_arima_4root	4	20.932534	32.67461	0.4896978	0.9500916
alpha	Base_arima_noTrans	1	10.439635	16.12521	0.6506410	0.9865385
alpha	Base_arima_noTrans	2	15.817662	24.53027	0.7060440	0.9931319
alpha	Base_arima_noTrans	3	19.082068	29.60696	0.7278107	0.9900676
alpha	Base_arima_noTrans	4	23.544684	36.95682	0.7152015	0.9890110
alpha	COVIDhub-baseline	1	13.308453	13.25449	0.9467949	0.9935897
alpha	COVIDhub-baseline	2	23.330555	24.64121	0.9609890	0.9945055
alpha	COVIDhub-baseline	3	27.315119	29.35524	0.9609045	0.9936602
alpha	COVIDhub-baseline	4	31.615696	35.64400	0.9558150	0.9945055
alpha	Multiple3_arima_4root	1	19.933671	28.37134	0.3147436	0.7115385
alpha	Multiple3_arima_4root	2	26.861465	35.38199	0.3458791	0.8401099
alpha	Multiple3_arima_4root	3	49.093669	40.97925	0.3609467	0.8683432
alpha	Multiple3_arima_4root	4	145.357628	53.06648	0.3326465	0.8566850
alpha	Multiple3_arima_noTrans	1	26.375280	39.81555	0.2814103	0.7891026
alpha	Multiple3_arima_noTrans	2	28.797458	49.04482	0.3277473	0.9675824
alpha	Multiple3_arima_noTrans	3	32.777879	56.29881	0.3560862	0.9771767
alpha	Multiple3_arima_noTrans	4	41.064825	70.59136	0.3331044	0.9771062
alpha	Topmost8_arima_4root	1	12.882355	19.40712	0.3878205	0.7993590
alpha	Topmost8_arima_4root	2	15.551326	24.84137	0.4107143	0.9068681
alpha	Topmost8_arima_4root	3	19.133755	30.76062	0.4110313	0.9285714
alpha	Topmost8_arima_4root	4	23.110194	37.05173	0.4068223	0.9358974
alpha	Topmost8_arima_noTrans	1	12.374814	19.69663	0.5628205	0.9660256

wave	model	horizon_wk	wis	mae	coverage_50	coverage_95
alpha	Topmost8_arima_noTrans	2	16.368797	24.14695	0.7024725	0.9887363
alpha	Topmost8_arima_noTrans	3	19.690620	29.79342	0.7035080	0.9917582
alpha	Topmost8_arima_noTrans	4	23.026581	35.65114	0.7163462	0.9956502
winter21	Base_arima_4root	1	27.337515	38.40443	0.4955128	0.8769231
winter21	Base_arima_4root	2	34.897533	53.75979	0.4692308	0.9105128
winter21	Base_arima_4root	3	43.764775	66.84763	0.4604396	0.8961538
winter21	Base_arima_4root	4	54.503134	82.35189	0.4406593	0.8706960
winter21	Base_arima_noTrans	1	30.227156	41.44114	0.3948718	0.8185897
winter21	Base_arima_noTrans	2	35.650477	53.21713	0.4569231	0.8664103
winter21	Base_arima_noTrans	3	44.022583	64.73021	0.4701465	0.8560440
winter21	Base_arima_noTrans	4	53.433066	77.64923	0.4738095	0.8388278
winter21	COVIDhub-baseline	1	23.898469	33.16410	0.7275641	0.9576923
winter21	COVIDhub-baseline	2	36.858143	52.51282	0.8348718	0.9930769
winter21	COVIDhub-baseline	3	44.295239	64.01593	0.8393773	0.9921245
winter21	COVIDhub-baseline	4	52.140461	76.78754	0.8388278	0.9899267
winter21	Multiple3_arima_4root	1	51.049101	65.91271	0.3198718	0.7282051
winter21	Multiple3_arima_4root	2	41.557763	62.21824	0.3735897	0.8641026
winter21	Multiple3_arima_4root	3	41.463801	63.32639	0.4197802	0.8981685
winter21	Multiple3_arima_4root	4	47.183514	72.72643	0.4184982	0.8868132
winter21	Multiple3_arima_noTrans	1	43.933403	55.86781	0.3493590	0.7339744
winter21	Multiple3_arima_noTrans	2	37.741169	56.35186	0.4133333	0.8925641
winter21	Multiple3_arima_noTrans	3	39.662756	60.76915	0.4538462	0.9075092
winter21	Multiple3_arima_noTrans	4	47.681638	73.82401	0.4423077	0.8946886
winter21	Topmost8_arima_4root	1	41.044347	54.98491	0.3769231	0.7775641
winter21	Topmost8_arima_4root	2	36.796953	56.15167	0.4179487	0.8971795
winter21	Topmost8_arima_4root	3	40.706065	62.73678	0.4362637	0.9021978
winter21	Topmost8_arima_4root	4	47.790747	73.28020	0.4399267	0.8853480
winter21	Topmost8_arima_noTrans	1	39.550685	51.12728	0.3647436	0.7576923
winter21	Topmost8_arima_noTrans	2	38.426954	58.16142	0.3715385	0.8692308
winter21	Topmost8_arima_noTrans	3	43.333986	66.69547	0.3840659	0.8721612
winter21	Topmost8_arima_noTrans	4	49.974749	76.42734	0.4141026	0.8575092

For US National during the alpha wave, the Base_arima_noTrans model performed the best for coverage rates and second best for WIS and MAE at every horizon. The Base_arima_4root model performed best in terms of WIS and MAE but was third for coverage rate while the Topmost8_arima_noTrans model had the second best coverage rates and third best WIS and MAE. During the alpha wave, these models performed the best and tended to have more consistent values for their metrics. The winter21 wave saw noticeably worse values for all metrics at every horizon. Generally, the two models that use the base set hierarchy tended to perform the best for all metrics, although the coverage rates varied a lot by horizon, with the Topmost8_arima_4root and Multiple3_arima_noTrans achieving closer to nominal coverage rates for the 95% interval than even the Base models at horizons greater than 1 week ahead.

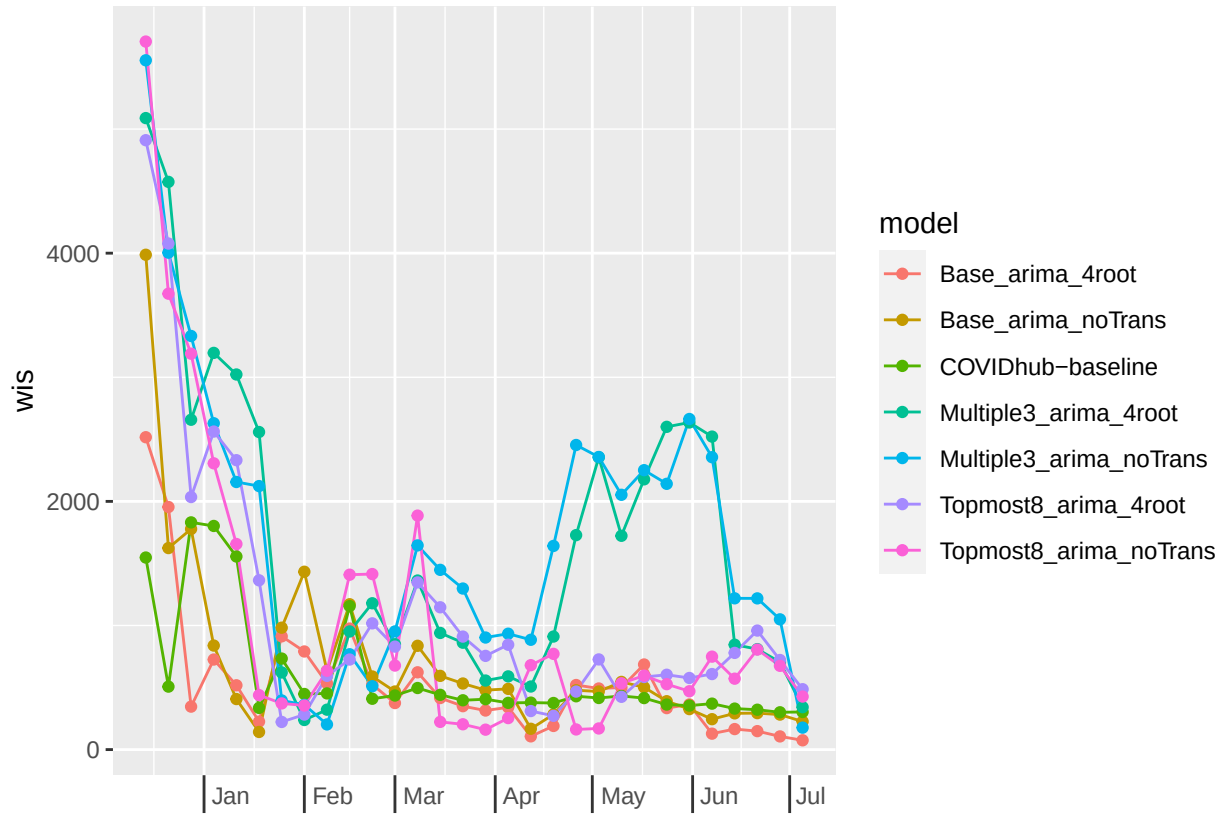
For states during both waves, we again see noticeably better coverage rates that approach the nominal level (or even exceed it) for all of the models, compared to those of the US National level when broken into waves. During the alpha wave, the Base_arima_4root model has the best performance for all metrics across all horizons. It is followed closely by the Topmost8_arima_noTrans and Base_arima_noTrans models, then the Topmost8_arima_4root and Multiple3_arima_noTrans models, all of which are much better than the Multiple3_arima_4root model. During the winter21 wave, the Base_arima_4root model is again generally the best for all metrics at all horizon weeks but sometimes loses out in terms of 95% coverage at longer horizons to other models. Notably during this wave, there are much greater similarities in coverage rates and certain models have better coverage rates for the 50% interval while others have better coverage reads for the 95% one.

By forecast date (week)

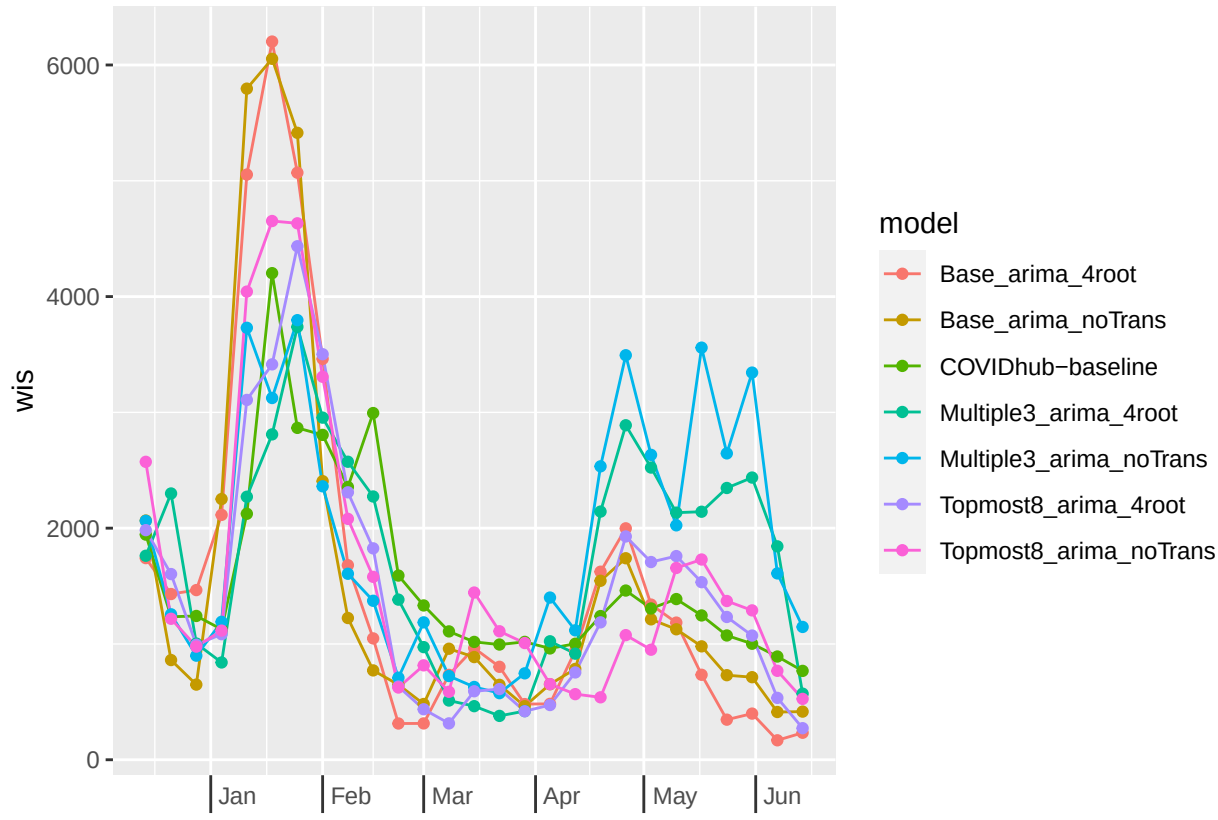
```
# US National
forecast_date_metrics_US <- scores_df %>%
  rbind(score_baseline) %>%
  filter(horizon <= 28,
         target_end_date <= as.Date("2021-07-05"),
         location == "US") %>%
  mutate(
    horizon_wk = case_when(
      horizon %in% c(1:7) ~ 1,
      horizon %in% c(8:14) ~ 2,
      horizon %in% c(15:21) ~ 3,
      horizon %in% c(22:28) ~ 4
    )
  ) %>%
  group_by(forecast_date, model, horizon_wk) %>%
  summarize(
    wis = mean(wis), mae=mean(abs_error),
    coverage_50 = mean(coverage_50),
    coverage_95 = mean(coverage_95)
  )
```

```
## `summarise()` has grouped output by 'forecast_date', 'model'. You can override using the `.groups` a
```

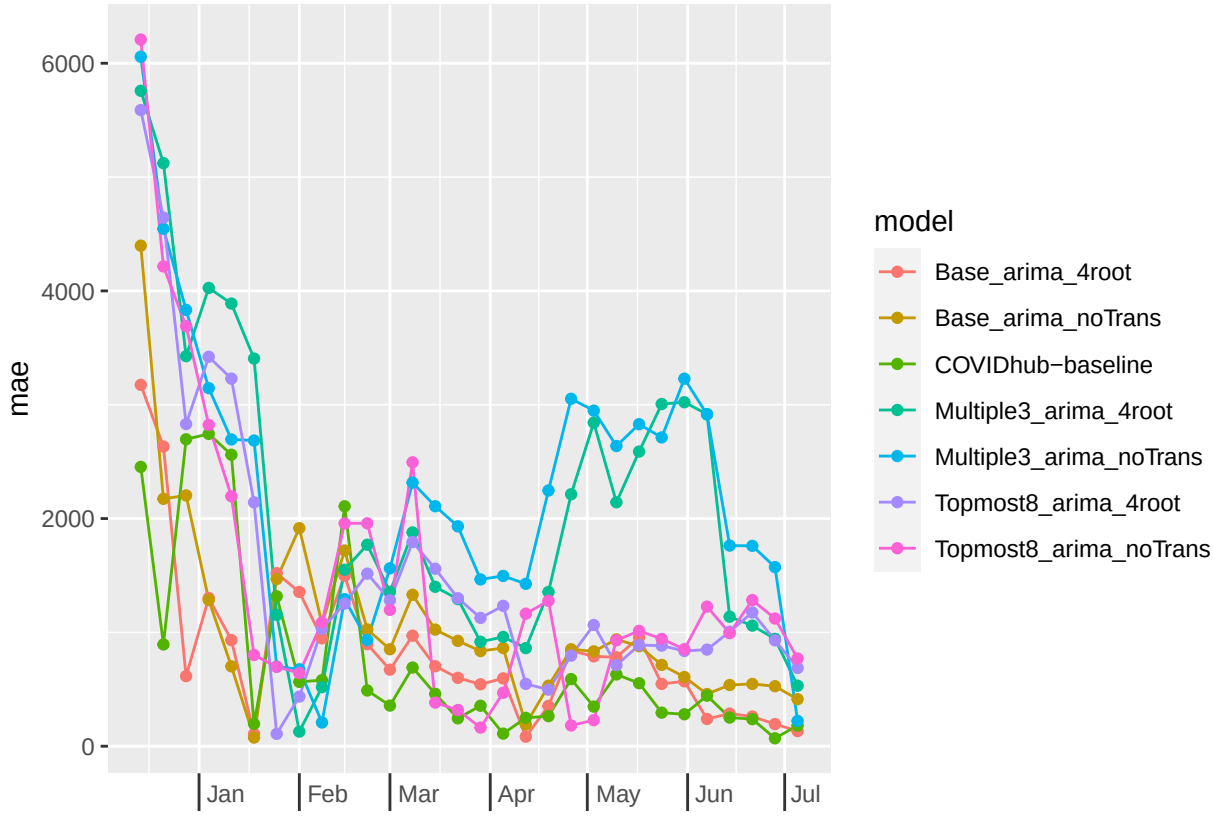
```
ggplot(filter(forecast_date_metrics_US, horizon_wk == 1),
       aes(x = forecast_date + weeks(1), y = wis)) +
  geom_line(aes(color=model)) +
  geom_point(aes(color=model)) +
  scale_x_date(name=NULL, date_breaks = "1 month", date_labels = "%b") +
  theme(axis.ticks.length.x = unit(0.5, "cm"),
    axis.text.x = element_text(vjust = 7, hjust = -0.2))
```



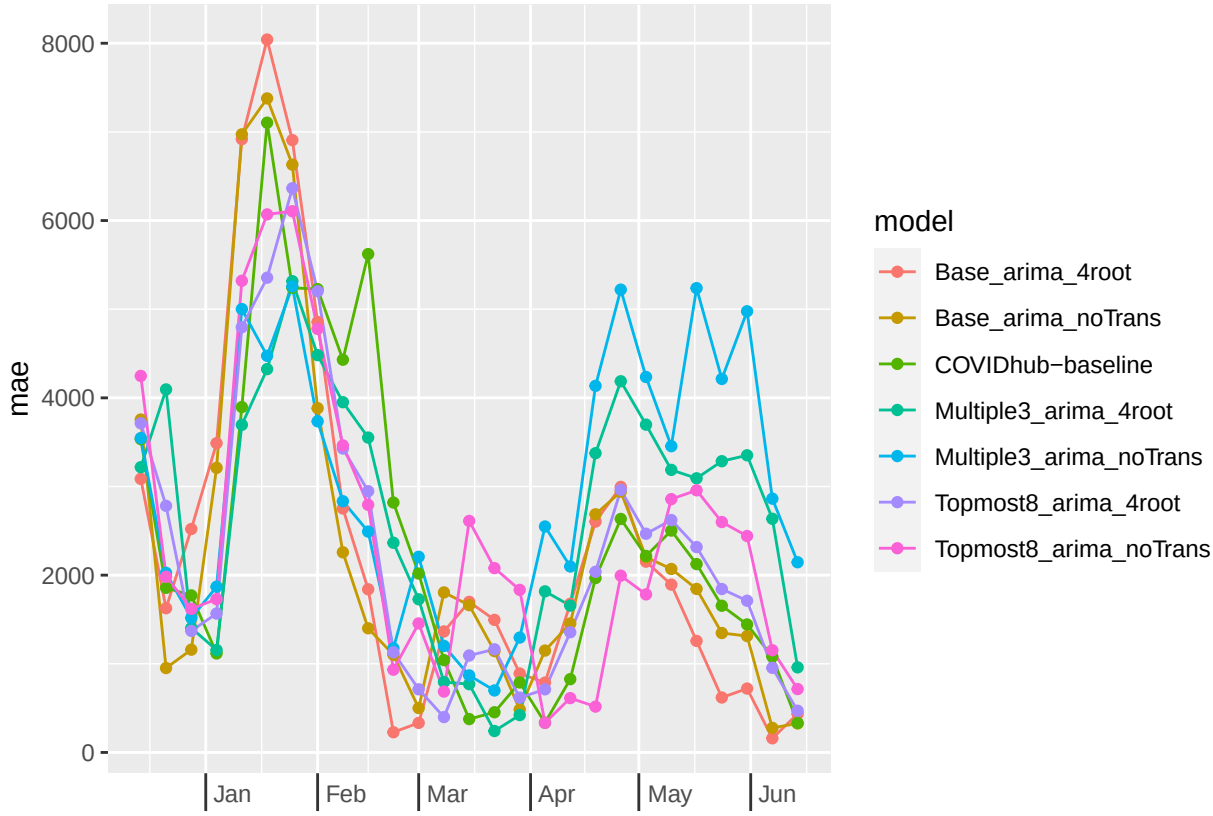

```
ggplot(filter(forecast_date_metrics_US, horizon_wk == 4),
       aes(x = forecast_date + weeks(1), y = wis)) +
  geom_line(aes(color=model)) +
  geom_point(aes(color=model)) +
  scale_x_date(name=NULL, date_breaks = "1 month", date_labels = "%b") +
  theme(axis.ticks.length.x = unit(0.5, "cm"),
    axis.text.x = element_text(vjust = 7, hjust = -0.2))
```




```
ggplot(filter(forecast_date_metrics_US, horizon_wk == 1),
       aes(x = forecast_date + weeks(1), y = mae)) +
  geom_line(aes(color=model)) +
  geom_point(aes(color=model)) +
  scale_x_date(name=NULL, date_breaks = "1 month", date_labels = "%b") +
  theme(axis.ticks.length.x = unit(0.5, "cm"),
    axis.text.x = element_text(vjust = 7, hjust = -0.2))
```




```
ggplot(filter(forecast_date_metrics_US, horizon_wk == 4),
       aes(x = forecast_date + weeks(1), y = mae)) +
  geom_line(aes(color=model)) +
  geom_point(aes(color=model)) +
  scale_x_date(name=NULL, date_breaks = "1 month", date_labels = "%b") +
  theme(axis.ticks.length.x = unit(0.5, "cm"),
    axis.text.x = element_text(vjust = 7, hjust = -0.2))
```

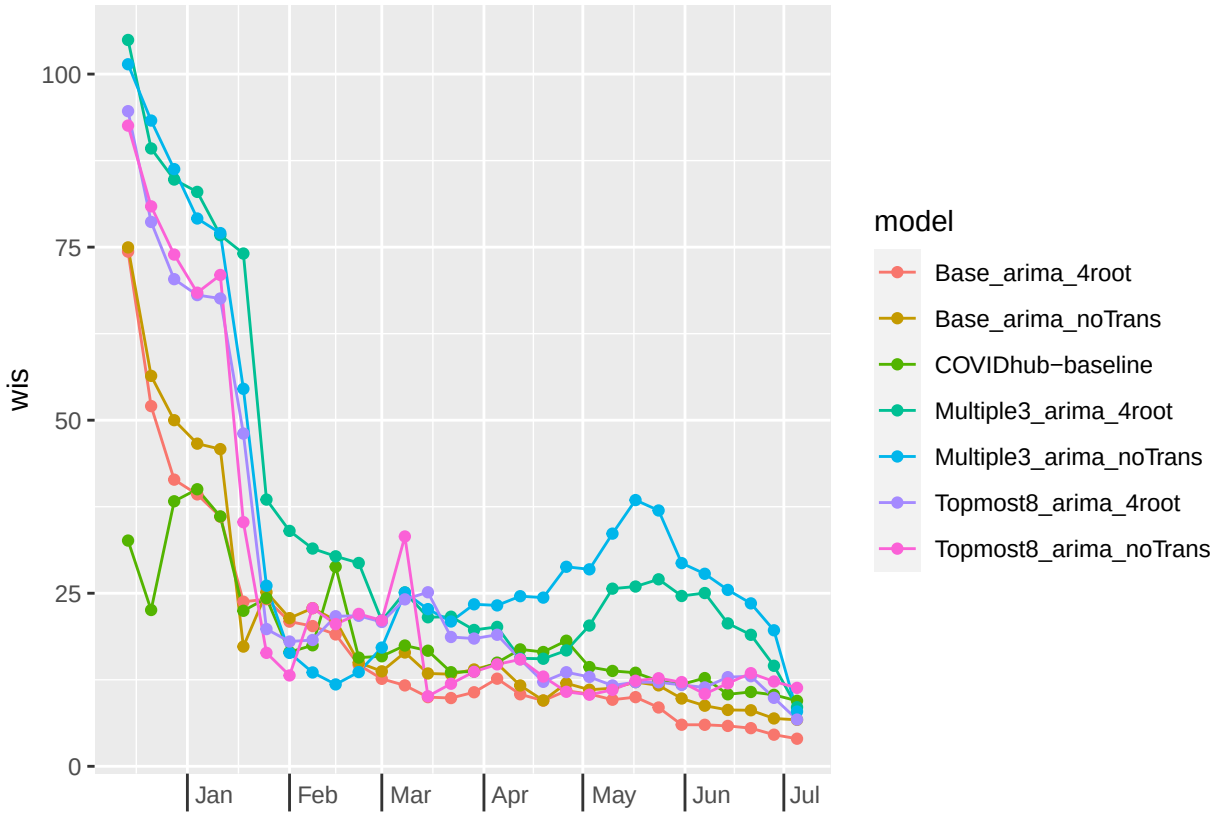



```
# Aggregated for all states and territories
forecast_date_metrics_states <- scores_df %>%
  rbind(score_baseline) %>%
  filter(horizon <= 28,
         target_end_date <= as.Date("2021-07-05"),
         location != "US") %>%
  mutate(
    horizon_wk = case_when(
      horizon %in% c(1:7) ~ 1,
      horizon %in% c(8:14) ~ 2,
      horizon %in% c(15:21) ~ 3,
      horizon %in% c(22:28) ~ 4
    )) %>%
  group_by(forecast_date, model, horizon_wk) %>%
  summarize(
    wis = mean(wis), mae=mean(abs_error),
    coverage_50 = mean(coverage_50),
    coverage_95 = mean(coverage_95)
  )
```

```
## `summarise()` has grouped output by 'forecast_date', 'model'. You can override using the `.groups` ar
```

```
ggplot(filter(forecast_date_metrics_states, horizon_wk == 1),
        aes(x = forecast_date + weeks(1), y = wis)) +
  geom_line(aes(color=model)) +
```

```
  geom_point(aes(color=model)) +
  scale_x_date(name=NULL, date_breaks = "1 month", date_labels = "%b") +
  theme(axis.ticks.length.x = unit(0.5, "cm"),
    axis.text.x = element_text(vjust = 7, hjust = -0.2))
```




```
#knitr::kable(forecast_date_metrics_states)
```

Scoring Conclusions

It seems that the best three models that should be passed onto the testing phase are the Base_arima_noTrans, the Base_arima_4root, and the Topmost8_arima_noTrans models, which seem to take turns being most accurate depending on state or national level overall and during different waves.